An Introduction to Complexity Theory

Computers execute tasks incredibly fast, yet many real-world problems remain too complex to solve as they are too time-consuming to solve using computers or too big to hold in them.

For example, multiplying two large matrices (10,000×10,000) can take over 16 minutes, even on a fast computer with a 2GHz CPU. Similarly, storing a large database, like 50 million names, can use more than 1GB of memory, which can slow down a computer.

Since computers have limited processing power and memory, programmers must use them wisely. To do this, they analyze two things:

- **Time complexity** (how long a program takes to run)
- **Space complexity** (how much memory it uses).

The time and memory a program needs can change depending on the input. However, we are most concerned with the worst-case scenario, the situation where the program takes the longest time or uses the most memory. This is called **worst-case complexity**. Worst-case complexity is important because it helps us understand the maximum resources a program might need and how long we may have to wait for the result.

Improving Worst Case Performance:

When searching for something in a large dataset, the way we search affects how long it takes. If we do not use an efficient method, the worst-case time can be very slow. Let us compare two common search methods: Linear Search and Binary Search to see how we can improve performance.

1. Linear Search (Slow, O(n))

Imagine you have a list of **1 million names**, and you are searching for **"Zara Khan"**

- With Linear Search, you start from the first name and check one by one until you find the right name.
- If "Zara Khan" is at the end, you have to check all 1 million names—this is the worst case.
- The more names on the list, the longer it takes.

2. Binary Search (Fast, O(log n))

Now, imagine the names are sorted alphabetically. Instead of checking one by one, you can use Binary Search:

- Open the list in the middle.
- If the middle name comes before "Zara Khan," ignore the first half and search only in the second half.
- Repeat this process, cutting the list in half each time.
- In the worst case, Binary Search only needs **about 20 steps** to find the name, instead of **1 million.**

Asymptotic Time/Space Complexity:

When we say the time/space complexity of an algorithm is proportional to some function of the input length N, suppose N^2, **the actual execution time is $C \times N^2$**, where C is a constant. The value of C depends on external factors like coding style, hardware efficiency, and data size. However, for complexity analysis, we usually ignore C because it becomes insignificant when analyzing worst-case complexity (when N becomes very large)

Let us examine the linear search and calculate the time complexity:

```
function linearSearch(arr, target):
        n = length(arr)  # 1 operation
        for i = 0 to n-1:  # n operations for incrementing loop n times
                if arr[i] == target:  # n operations for comparing n times
                        return i  # 1 operation if target is found
        return -1  # 1 operation is target is not found
```

The number of operations is 2N + 2.
When analyzing complexity, we ignore constants (like 2 and +2).
The dominant term is N, so the time complexity is O(N).

The formal way to express the worst-case running time and space complexity as a function of the input size as the input grows to infinity is to use the **big O notation**. So, we say the time complexity of linear search is O(n), and binary search is $O(\log_2 n)$. These are called asymptotic worst-case time complexities because they drop all constant multipliers and constant factor addition to the running time.

Space complexity refers to the amount of memory an algorithm uses relative to the input size N. It includes space for variables and auxiliary structures. For example, Linear Search has O(1) space complexity because it only needs a few extra variables, regardless of input size. **Now, suppose you need to make a copy of the entire list before searching. This means the memory used grows with the size of the list - this is O(n) space complexity.**

Note that when analyzing a program's time and space complexity, **we also drop lower-order terms in polynomial functions**. For example, if the complexity is $an^3 + bn^2 + cn + d$, we express it as $O(n^3)$ since, for large n, the highest power dominates. Lower powers become insignificant compared to the leading term.

Follow-up Question:

Q) What is the time and space complexity of the following pseudocode?

```
function createArray(n):
    array = new Array[n]  # Create a 1D array of size n
    for i = 0 to n-1:
        for j = 0 to n-1:
            array[i] = i + j
    return array
```

Ans: Time complexity is $O(n^2)$ due to nested loops and each loop runs n times.
Space complexity is O(n) due to the creation of an array of n length.

Q) What is the time complexity of Python's built-in "max" function?

Ans: O(n) since behind the built-in function, there is a single loop to find the maximum value.